

Lab05_DiegoLeite

Diego Leite de Souza - 182745

Crie uma função que: a. Receba um valor de TAIL_NUMBER (por exemplo, N431WN); b. Produza uma tabela (tidy) com todos os trajetos realizados pela aeronave (ordenadas por data e hora, contendo todas as colunas do arquivo flights.csv.zip); c. Produza um mapa que apresente todo o trajeto voado pela aeronave ao longo de todo o ano; o trajeto deve ser apresentado de maneira linear no tempo (i.e., segue a sequência do tempo, como no exemplo hipotético dado acima); d. O mapa deve ser decorado com estatísticas do seu interesse (por exemplo, velocidade média do voo como espessura da linha que conecta os aeroportos envolvidos no trajeto)

Primeiramente, criando a função principal

```
library(tidyverse)
```

Warning: pacote 'ggplot2' foi compilado no R versão 4.5.3

```
— Attaching core tidyverse packages ————— tidyverse 2.0.0
—
✓ dplyr      1.1.4      ✓ readr      2.1.5
✓ forcats    1.0.0      ✓ stringr    1.5.1
✓ ggplot2    4.0.3      ✓ tibble     3.2.1
✓ lubridate  1.9.4      ✓ tidyr      1.3.1
✓ purrr      1.0.4
— Conflicts ————— tidyverse_conflicts()
—
* dplyr::filter() masks stats::filter()
* dplyr::lag()     masks stats::lag()
i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all
conflicts to become errors
```

```
library(lubridate)
library(leaflet)
```

Warning: pacote 'leaflet' foi compilado no R versão 4.5.3

```
library(scales)
```

Warning: pacote 'scales' foi compilado no R versão 4.5.3

Anexando pacote: 'scales'

O seguinte objeto é mascarado por 'package:purrr':

discard

O seguinte objeto é mascarado por 'package:readr':

col_factor

```
analisa_aeronave <- function(tail_number, arquivo) {  
  
  # 1. Leitura dos aeroportos  
  caminho_aeroportos <- file.path("dados", "airports.csv")  
  if (!file.exists(caminho_aeroportos)) {  
    stop("O arquivo 'airports.csv' não foi encontrado na pasta 'dados'.")  
  }  
  tabela_aeroportos <- read_csv(caminho_aeroportos, show_col_types = FALSE)  
  
  # 2. Leitura dos voos em chunks  
  filtra_aeronave <- function(x, pos) {  
    x %>% filter(TAIL_NUMBER == tail_number)  
  }  
  
  tabela_voos_bruta <- read_csv_chunked(  
    file = arquivo,  
    callback = DataFrameCallback$new(filtra_aeronave),  
    chunk_size = 1e6,  
    show_col_types = FALSE  
  )  
  
  if (nrow(tabela_voos_bruta) == 0) {  
    stop(paste("Nenhum registro encontrado para a aeronave:", tail_number))  
  }  
  
  # 3. Tratamento dos dados e ordenação cronológica  
  tabela_voos_tidy <- tabela_voos_bruta %>%  
    filter(!is.na(DEPARTURE_TIME), !is.na(AIR_TIME), AIR_TIME > 0) %>%  
    mutate(  
      dep_time_num = as.numeric(DEPARTURE_TIME),  
      hora = dep_time_num %/% 100,  
      minuto = dep_time_num %% 100,  
      hora = ifelse(hora == 24, 0, hora),  
    )  
}
```

```

    data_hora_partida = make_datetime(YEAR, MONTH, DAY, hora, minuto),
    velocidade_media_mph = (DISTANCE / AIR_TIME) * 60
  ) %>%
  filter(!is.na(data_hora_partida)) %>%
  arrange(data_hora_partida)

# 4. Cruzamento com coordenadas
tabela_voos_coords <- tabela_voos_tidy %>%
  left_join(tabela_aeroportos %>% select(IATA_CODE, LATITUDE, LONGITUDE), by
= c("ORIGIN_AIRPORT" = "IATA_CODE")) %>%
  rename(lat_origem = LATITUDE, lon_origem = LONGITUDE) %>%
  left_join(tabela_aeroportos %>% select(IATA_CODE, LATITUDE, LONGITUDE), by
= c("DESTINATION_AIRPORT" = "IATA_CODE")) %>%
  rename(lat_destino = LATITUDE, lon_destino = LONGITUDE) %>%
  filter(!is.na(lat_origem), !is.na(lat_destino))

# 5. CONSTRUÇÃO DO MAPA
paleta_velocidade <- colorNumeric("YlOrRd", domain =
tabela_voos_coords$velocidade_media_mph)

aeroportos_unicos <- bind_rows(
  tabela_voos_coords %>% select(codigo = ORIGIN_AIRPORT, lat = lat_origem, lon
= lon_origem),
  tabela_voos_coords %>% select(codigo = DESTINATION_AIRPORT, lat = lat_destino,
lon = lon_destino)
) %>% distinct(codigo, .keep_all = TRUE)

mapa_trajetoria <- leaflet() %>%
  addProviderTiles(providers$CartoDB.Positron) %>%
  addCircleMarkers(
    data = aeroportos_unicos,
    lng = ~lon, lat = ~lat,
    radius = 3, color = "#008080", stroke = FALSE, fillOpacity = 0.8,
    popup = ~codigo
  )

# Adiciona as linhas usando scales e rescale para manter espessura fina
intervalo_velocidade <- range(tabela_voos_coords$velocidade_media_mph)

for (i in seq_len(nrow(tabela_voos_coords))) {
  voo <- tabela_voos_coords[i, ]
  espessura_fina <- rescale(voo$velocidade_media_mph, to = c(0.8, 2.5), from
= intervalo_velocidade)

  mapa_trajetoria <- mapa_trajetoria %>%
    addPolylines(
      lng = c(voo$lon_origem, voo$lon_destino),
      lat = c(voo$lat_origem, voo$lat_destino),

```

```

    weight = espessura_fina,
    opacity = 0.35,
    color = paleta_velocidade(voo$velocidade_media_mph),
    popup = paste0("Voo ", i, ": ", voo$ORIGIN_AIRPORT, " → ",
voo$DESTINATION_AIRPORT, " (", round(voo$velocidade_media_mph), " mph)")
  )
}

mapa_trajetoria <- mapa_trajetoria %>%
  addLegend(
    pal = paleta_velocidade,
    values = tabela_voos_coords$velocidade_media_mph,
    title = "Velocidade (mph)",
    position = "bottomright"
  )

return(list(
  tabela = tabela_voos_tidy,
  grafico = mapa_trajetoria
))
}

```

Agora aplicando a função

```

resultado <- analisa_aeronave(
  tail_number = "N431WN",
  arquivo = file.path("dados", "flights.csv.zip")
)

```

```
resultado$tabela
```

```

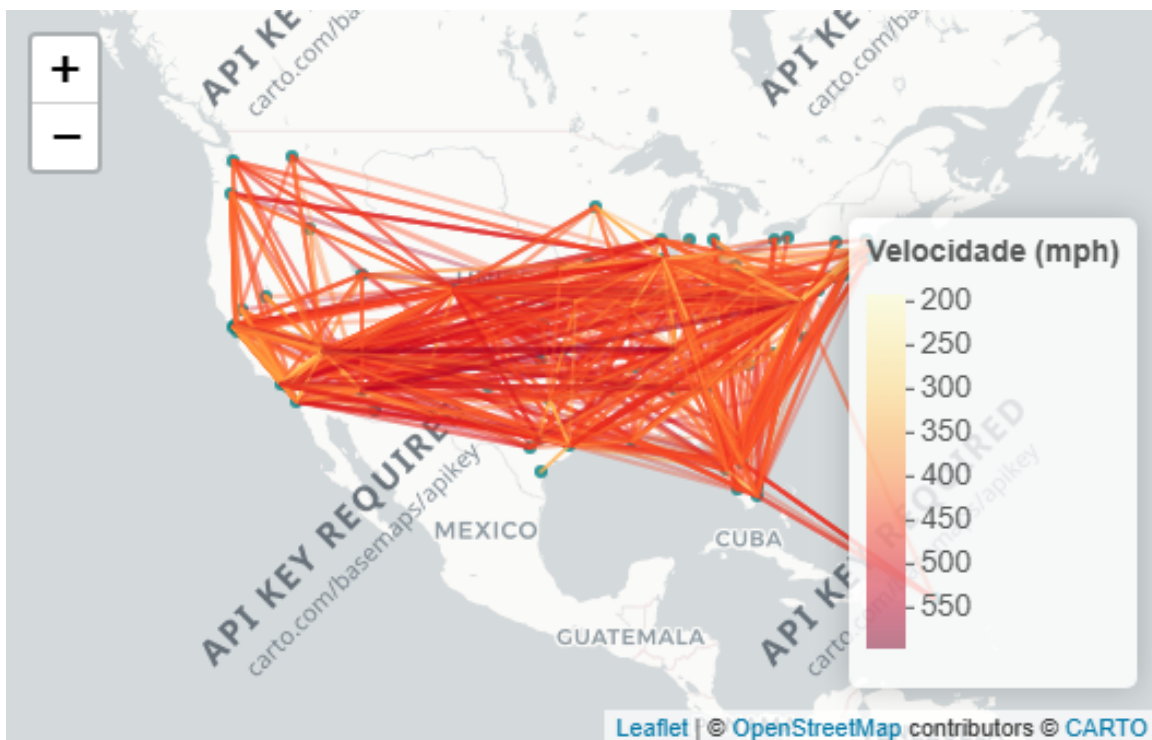
# A tibble: 1,921 × 36
  YEAR MONTH DAY DAY_OF_WEEK AIRLINE FLIGHT_NUMBER TAIL_NUMBER
  <dbl> <dbl> <dbl>      <dbl> <chr>      <dbl> <chr>
1  2015     1     1           4 WN          1762 N431WN
2  2015     1     1           4 WN          3564 N431WN
3  2015     1     1           4 WN          3564 N431WN
4  2015     1     1           4 WN          3564 N431WN
5  2015     1     2           5 WN          1793 N431WN
6  2015     1     2           5 WN          1793 N431WN
7  2015     1     2           5 WN          1793 N431WN
8  2015     1     2           5 WN          1793 N431WN
9  2015     1     2           5 WN          1793 N431WN
10 2015     1     2           5 WN          3144 N431WN
# i 1,911 more rows
# i 29 more variables: ORIGIN_AIRPORT <chr>, DESTINATION_AIRPORT <chr>,

```

```
# SCHEDULED_DEPARTURE <chr>, DEPARTURE_TIME <chr>, DEPARTURE_DELAY <dbl>,
# TAXI_OUT <dbl>, WHEELS_OFF <chr>, SCHEDULED_TIME <dbl>, ELAPSED_TIME <dbl>,
# AIR_TIME <dbl>, DISTANCE <dbl>, WHEELS_ON <chr>, TAXI_IN <dbl>,
# SCHEDULED_ARRIVAL <chr>, ARRIVAL_TIME <chr>, ARRIVAL_DELAY <dbl>,
# DIVERTED <dbl>, CANCELLED <dbl>, CANCELLATION_REASON <chr>, ...
```

```
resultado$grafico
```

```
file:///C:/Users/Windows/AppData/Local/Temp/Rtmp2bVfMV/file4008311567ab/
widget40081ece34e4.html screenshot completed
```



horário de compilação

```
Sys.setenv(TZ = "America/Sao_Paulo")
Sys.time()
```

```
[1] "2026-09-17 00:27:45 -03"
```